UCT305: SOFTWARE ENGINEERING

L	T	P	Cr
3	0	2	4.0

Course Objectives: To understand, analyze, specify and manage software requirements by applying principles of software development and evolution. To design, implement and test software using object oriented principles and quality standards.

Introduction: Programming in the small vs. programming in the large; software project failures and importance of software quality and timely availability; engineering approach to software development; role of software engineering towards successful execution of large software projects; emergence of software engineering as a discipline.

Software Project Management: Basic concepts of life cycle models – different models and milestones; software project planning –identification of activities and resources; concepts of feasibility study; techniques for estimation of schedule and effort; software cost estimation models and concepts of software engineering economics; techniques of software project control and reporting; introduction to measurement of software size; introduction to the concepts of risk and its mitigation; configuration management.

Software Quality and Reliability: Internal and external qualities; process and product quality; principles to achieve software quality; introduction to different software quality models like McCall, Boehm, FURPS / FURPS+, Dromey, ISO – 9126; introduction to Capability Maturity Models (CMM and CMMI); introduction to software reliability, reliability models and estimation.

Software Requirements Analysis, Design and Construction: Introduction to Software Requirements Specifications (SRS) and requirement elicitation techniques; techniques for requirement modeling – decision tables, event tables, state transition tables, Petri nets; requirements documentation through use cases; introduction to UML, introduction to software metrics and metrics based control methods; measures of code and design quality.

Object Oriented Analysis, Design and Construction: Concepts -- the principles of abstraction, modularity, specification, encapsulation and information hiding; concepts of abstract data type; Class Responsibility Collaborator (CRC) model; quality of design; design measurements; concepts of design patterns; Refactoring; object oriented construction principles; object oriented metrics.

Software Testing: Introduction to faults and failures; basic testing concepts; concepts of verification and validation; black box and white box tests; white box test coverage – code coverage, condition coverage, branch coverage; basic concepts of black-box tests – equivalence classes, boundary value tests, usage of state tables; testing use cases; transaction based testing; testing for non-functional requirements – volume, performance and efficiency; concepts of inspection.

Laboratory Work:

Development of requirements specification, function oriented design using SA/SD, object-oriented design using UML, test case design, implementation using C++ and testing. Use of appropriate CASE tools and other tools such as configuration management tools, program analysis tools in the software life cycle.

Course Learning Outcomes (CLOs) / Course Objectives (COs):

Upon completion of this course, students will be able to:

- Analyze software requirements through various tools like DFD and Use Case diagrams for creating SRS.
- Develop software design model using Object oriented analysis and design concepts.
- Apply testing techniques to test the software functionality and non-functional requirements.
- Achieve Software quality through quality standards and Models.
- Demonstrate project management by software planning and estimations.

Text Books:

1. Software Engineering, Ian Sommerville.

Reference Books:

1. *Fundamentals of Software Engineering,* Carlo Ghezzi, Jazayeri Mehdi, Mandrioli Dino
2. *Software Requirements and Specification: A Lexicon of Practice, Principles and Prejudices*, Michael Jackson
3. *The Unified Development Process*, Ivar Jacobson, Grady Booch, James Rumbaugh
4. *Design Patterns: Elements of Object-Oriented Reusable Software*, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
5. *Software Metrics: A Rigorous and Practical Approach*, Norman E Fenton, Shari Lawrence Pfleeger
6. *Software Engineering: Theory and Practice*, Shari Lawrence Pfleeger and Joanne M. Atlee
7. *Object-Oriented Software Construction,* Bertrand Meyer
8. *Object Oriented Software Engineering: A Use Case Driven Approach* --Ivar Jacobson
9. *Touch of Class: Learning to Program Well with Objects and Contracts* --Bertrand Meyer
10. *UML Distilled: A Brief Guide to the Standard Object Modeling Language* --Martin Fowler.